

Kubernetes and Cloud Native Associate - KCNA

Cloud Native Architecture

Open Standards

In this article, we explore the concept of Open Standards and their transformative impact on cloud-native technologies.

Imagine you have a mobile phone that you want to use while traveling. When abroad, you might need a different charger or adapter because countries use different plug and socket designs. Similarly, in the cloud-native ecosystem, various services—such as containers, orchestration tools, networking, and storage—must seamlessly interact. Without a set of universal guidelines, these technologies risk compatibility issues and vendor lock-in.

Open Standards are publicly available specifications, protocols, or formats developed collaboratively through consensus. They promote interoperability, portability, and vendor neutrality, ensuring that diverse technologies can work together smoothly.

| Open Standards

Interoperability
Portability
Vendor Neutrality



Consider the analogy of standardized electric plugs and sockets. Many countries have adopted a few common designs, such as type A, B, C, and D plugs. If your charger uses a type A plug—common in the US and Japan—you can easily use it in countries that support the same type, eliminating the need for additional adapters. Although a single global standard would be ideal, the current standardization still serves us well.

| Analogy



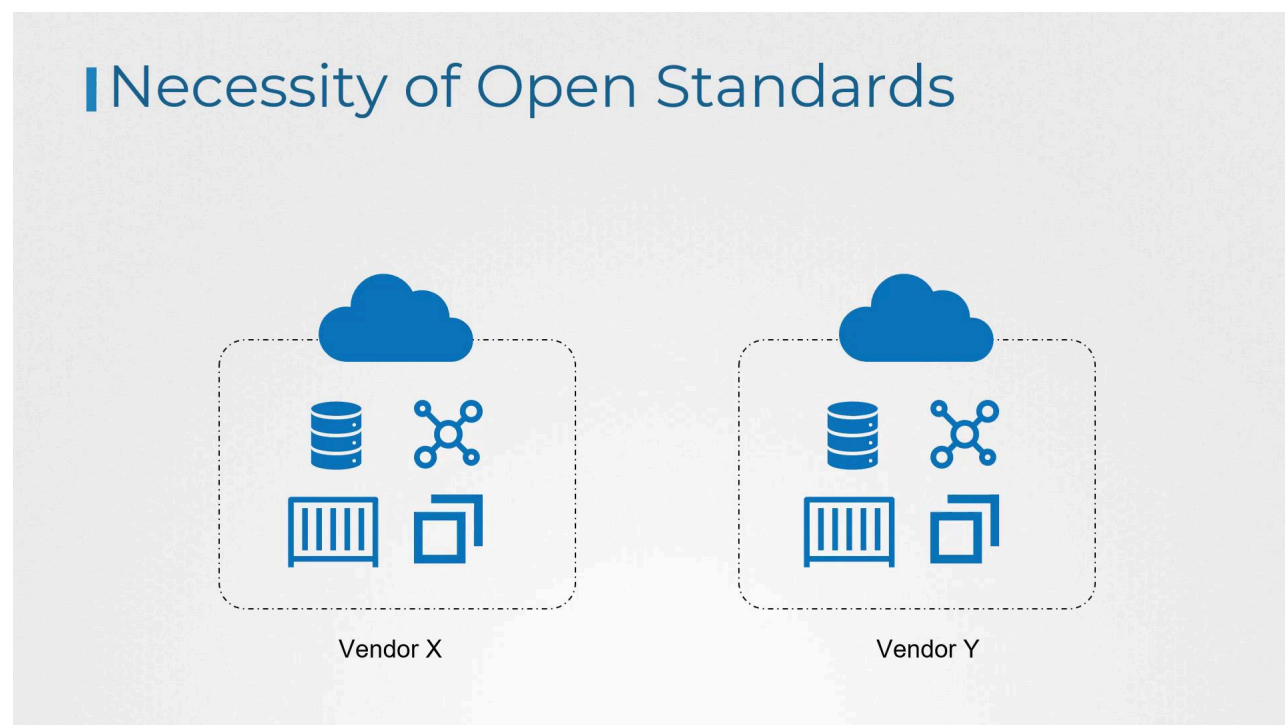
USA



Japan

In the cloud-native environment, adopting Open Standards enables different technologies to interact effortlessly regardless of their vendor origin. This interoperability empowers developers to integrate a variety of technologies and services into their applications without significant modifications. Moreover, Open Standards provide the flexibility to switch between technologies, minimizing dependency on a single vendor and fostering healthy competition and innovation.

Consider a scenario where you develop a containerized application using a specific container format and runtime from Vendor X. Later, if you decide to migrate your application to a cloud provider that uses a different container format and runtime from Vendor Y, Open Standards ensure that you can seamlessly transition without extensive rewriting or reconfiguration.



The fragmentation within the container ecosystem necessitates a common language—enter Open Standards.

Open Container Initiatives (OCI)



Note

The Open Container Initiative (OCI) is a key organization dedicated to developing open standards for container images, runtimes, and distributions.

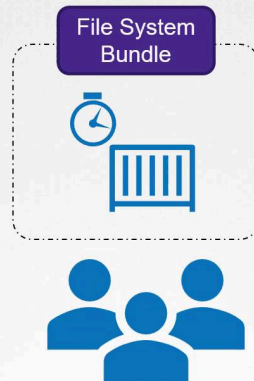
The Role of OCI in Container Technology

OCI has introduced standards that have revolutionized container technology:

1. Image Specification

This standard defines how a filesystem bundle should be packaged into an image. As a result, various build tools—like BuildKit, Podman, Buildah, and Docker—can create OCI-compliant images.

Open Container Initiatives (OCI)



2. Container Runtime Specification

This standard outlines how to download, unpack, and run the filesystem bundle using any OCI-compliant runtime. While Docker was once the primary runtime, alternatives such as ContainerD, CRI, Kata Containers, gVisor, and Firecracker now offer robust options.

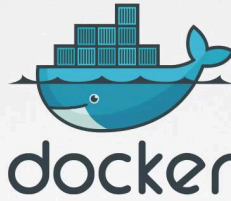
3. Distribution Specification

This defines standardized protocols for distributing container images. While Docker Hub was originally the dominant platform, other repositories like Amazon ECR and Microsoft Azure now conform to this standard.

| Distribution Specification



Amazon Elastic
Container Registry(ECR)



Microsoft
Azure

Kubernetes and Open Standards

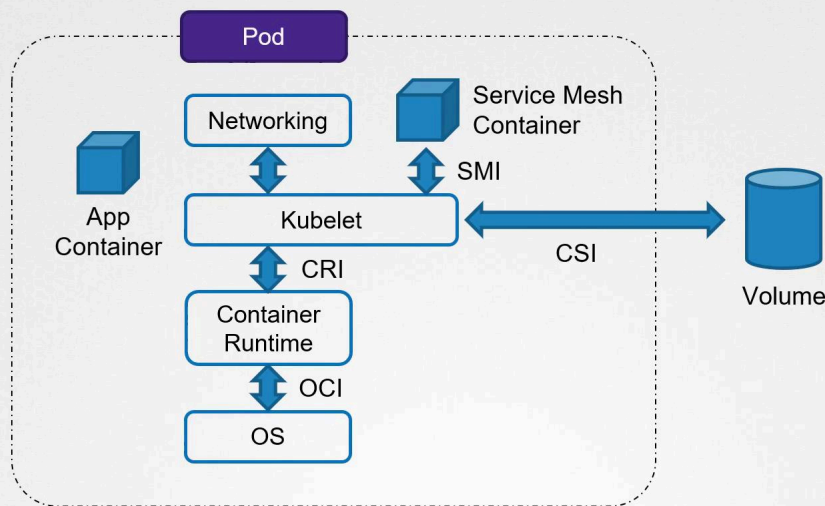
Kubernetes is a strong proponent of open standards and modularity. By supporting pluggable layers for runtime, networking, service meshes, and storage, Kubernetes offers a flexible environment for building cloud-native infrastructures. This decoupling enables users to assemble the best components from various vendors.

Below are some key Kubernetes interfaces:

1. Container Runtime Interface (CRI)

The CRI is an open standard that provides a pluggable container runtime layer. It allows users to choose the optimal runtime for their needs, even permitting different nodes within a cluster to use distinct container runtimes. Initially, Kubernetes relied on Docker as its runtime, but many clusters now default to ContainerD, which is CRI-compatible while Docker is not.

Kubernetes Open Standards



Container Runtime Interface (CRI)



2. Container Network Interface (CNI)

CNI provides a standard interface for Kubernetes network plugins to configure container networking. This standard enables features such as network policies, service discovery, and load balancing, while also facilitating connections to virtual networks and third-party networking solutions.

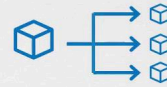
| Container Network Interface (CNI)



Network
Policies



Service Discovery



Load
Balancing



Container Network
Interface

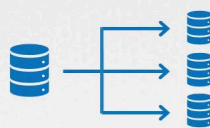
3. Container Storage Interface (CSI)

CSI enables Kubernetes to work with various storage solutions, including cloud storage, network-attached storage (NAS), and storage area networks (SAN). By separating core Kubernetes functions from storage implementations, third-party providers can develop plugins that integrate seamlessly, supporting dynamic provisioning and management of storage volumes.

| Container Storage Interface (CSI)



Cloud
Storage



Network-attached Storage

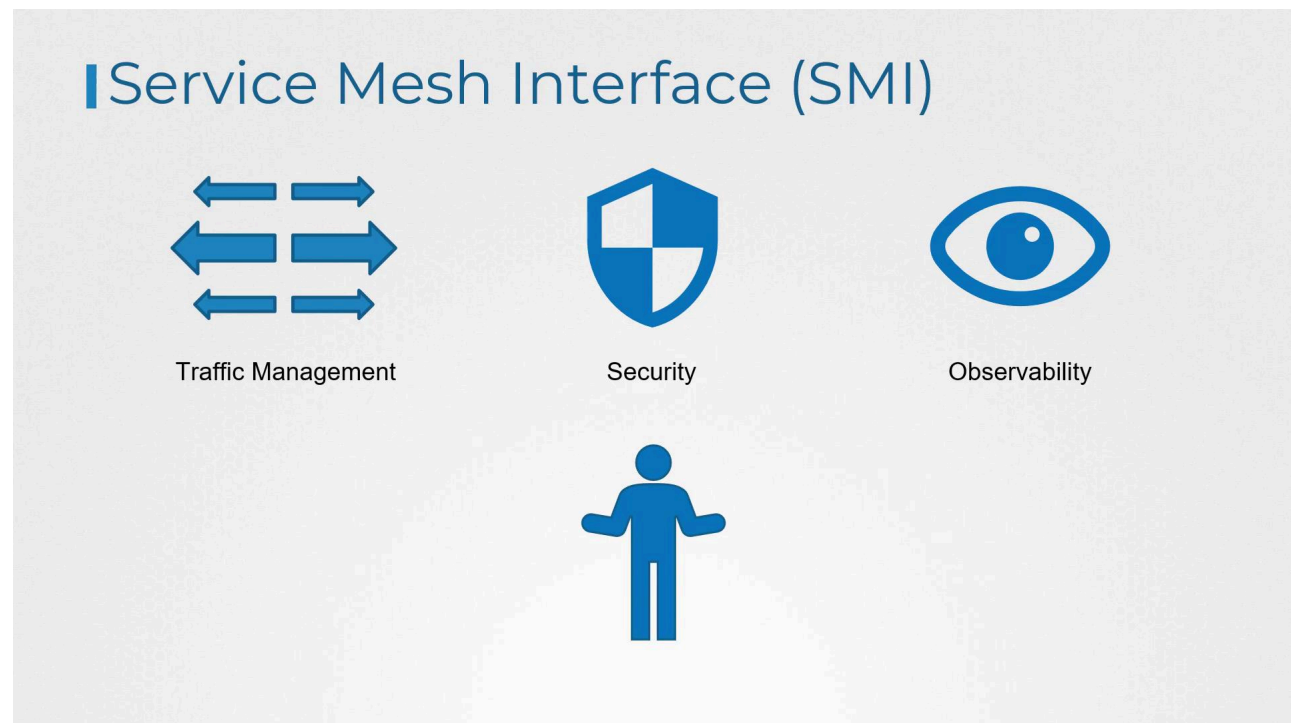


Storage Area Networks



4. Service Mesh Interface (SMI)

SMI standardizes the interaction between service mesh components, offering a set of APIs that ensure seamless communication between control planes and data planes. This vendor-agnostic specification helps manage traffic, enhance security, and improve observability across service mesh deployments.



Additional Resources

For those interested in exploring further, the following resources provide deeper insights into open standards in cloud-native technologies:

- [CloudNative Community Foundation \(CNCF\)](#)
- [Open Container Initiative \(OCI\)](#)

Thank you for reading this article. Enjoy exploring the world of Open Standards and cloud-native technologies, and we look forward to your next visit!



Watch Video

Watch video content

Previous

◀ Kubernetes SIG

Next

Observability Fundamentals ▶



Beta



● Search docs...



Kubernetes and Cloud Native Associate - KCNA ▼